PROGRAMMING FOR PROBLEM SOLVING (3110003)

Prepared By,

Prof. Shraddha Modi

Computer Engineering

LDCE, Ahmedabad

CHAPTER-3
CONTROL STRUCTURE IN C

Prepared By,
Prof. Shraddha Modi
Computer Engineering
LDCE, Ahmedabad

ROADMAP

- Simple statements, Decision making statements/conditional statements
- Looping statements, Nesting of control structures,
- Break and continue,
- Goto statement

INTRODUCTION

- In C, statements are executed sequentially.
- C provides two styles of flow control to change the sequential execution of program:
  - Branching (Decision Making)
  - Looping
- Branching is deciding **what actions to take** and looping is deciding **how many times to take a certain action**.

DECISION MAKING STATEMENT

- C language provides such decision making capabilities by supporting “IF” statement.
- Types of “IF” statements:
  - Simple if statement
  - if...else statement
  - Nested if...else statement
  - else if ladder

SIMPLE IF

- Used to execute or skip one statement or group of statements for a particular condition.

- **Syntax:**

```
if (condition)
{
Block of statements;
 }
Next Statement X;
```

If the condition is true then the statement or block of statements gets executed otherwise these statements are skipped and Statement X will be executed

SIMPLE IF

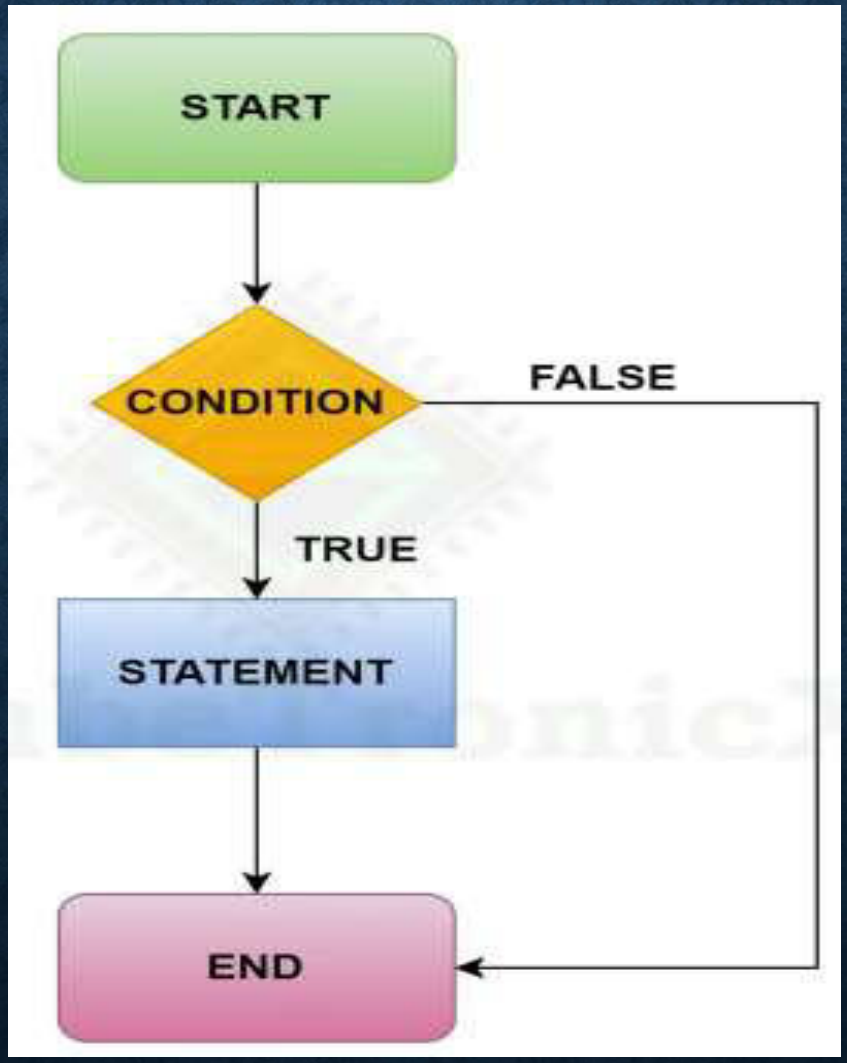
- To execute only one statement, opening and closing braces are not required.

```
if(a >= b)
{
    printf("%d is largest", a);
}
```

Both are same

```
if(a >= b)
    printf("%d is largest", a);
```

SIMPLE IF



Statement-x

SIMPLE IF PROGRAM

```
#include<stdio.h>
        void main()
         {
          int i = 5;
          if (i >1)
            {
              printf(" i is greater than 1\n");
             }
         printf(" End of program\n");
        }
```

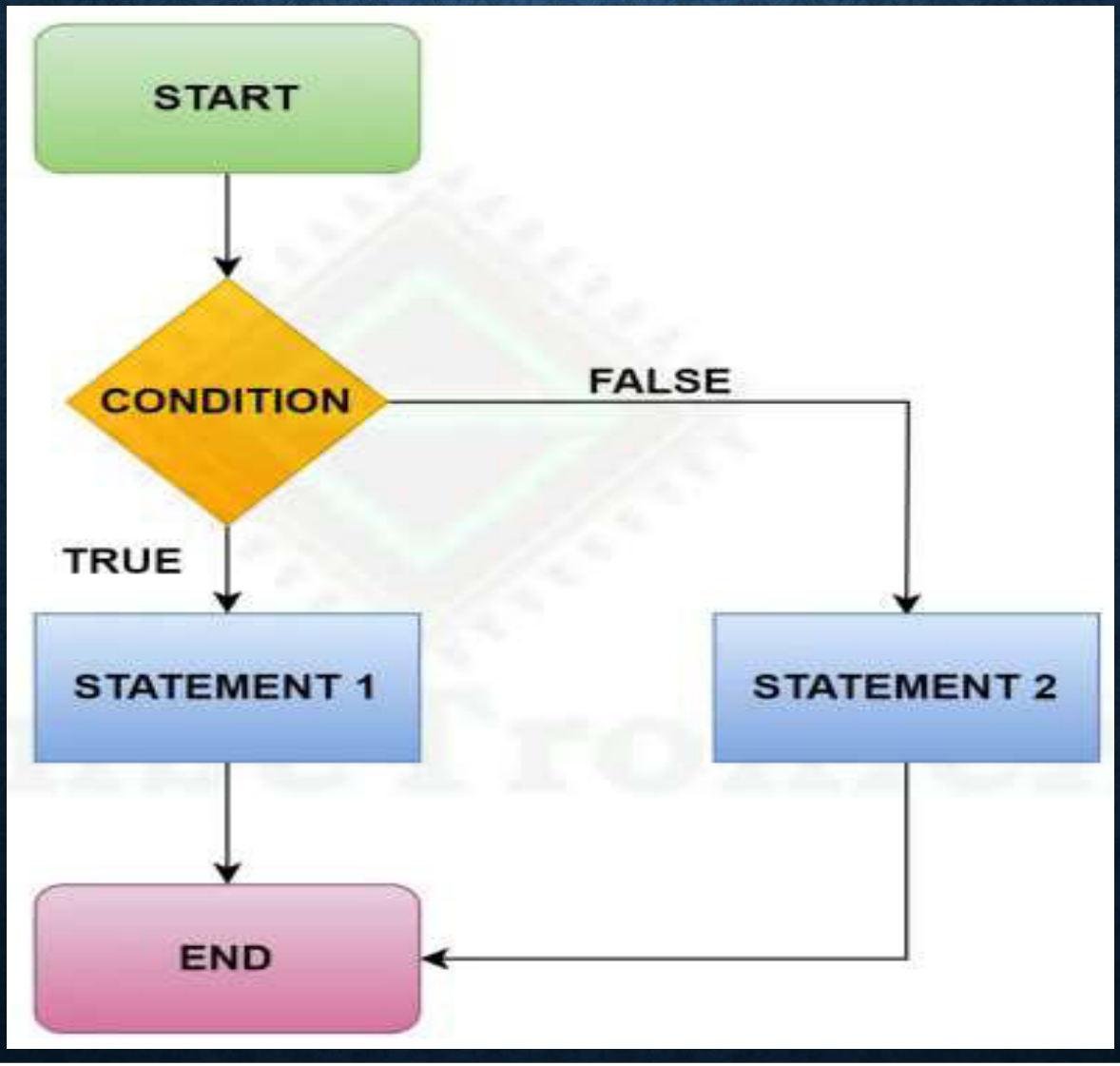
SIMPLE IF ELSE STATEMENT

- The if....else statement is an extension of the simple if statement.
- Syntax:

```
if (condition)
 {
   Block of statements-1;
}
else
{
Block of statements-2;
}
 Next Statement;
```

If the condition is true then the block of statements-1 gets executed otherwise Block of Statement-2 will be executed

SIMPLE IF ELSE STATEMENT FLOWCHART



SIMPLE IF ELSE STATEMENT PROGRAM

```c
#include <stdio.h>

int main()
{
   int num;
   printf ("Enter an number: ");
   scanf ("%d", &num);
   //if the remainder is 0 (true)
   if (num %2 == 0)
   {
     printf ("%d is an even number.",num);
   }
   else
   {
     printf ("%d is an odd number.",num);
   }
   printf("Goodbye!!!");
   return 0;
}
```

```
Enter an number: 7
7 is an odd number.
Goodbye!!!
```

```
Enter an number: 4
4 is an even number.
Goodbye!!!
```

SIMPLE IF ELSE STATEMENT PROGRAM

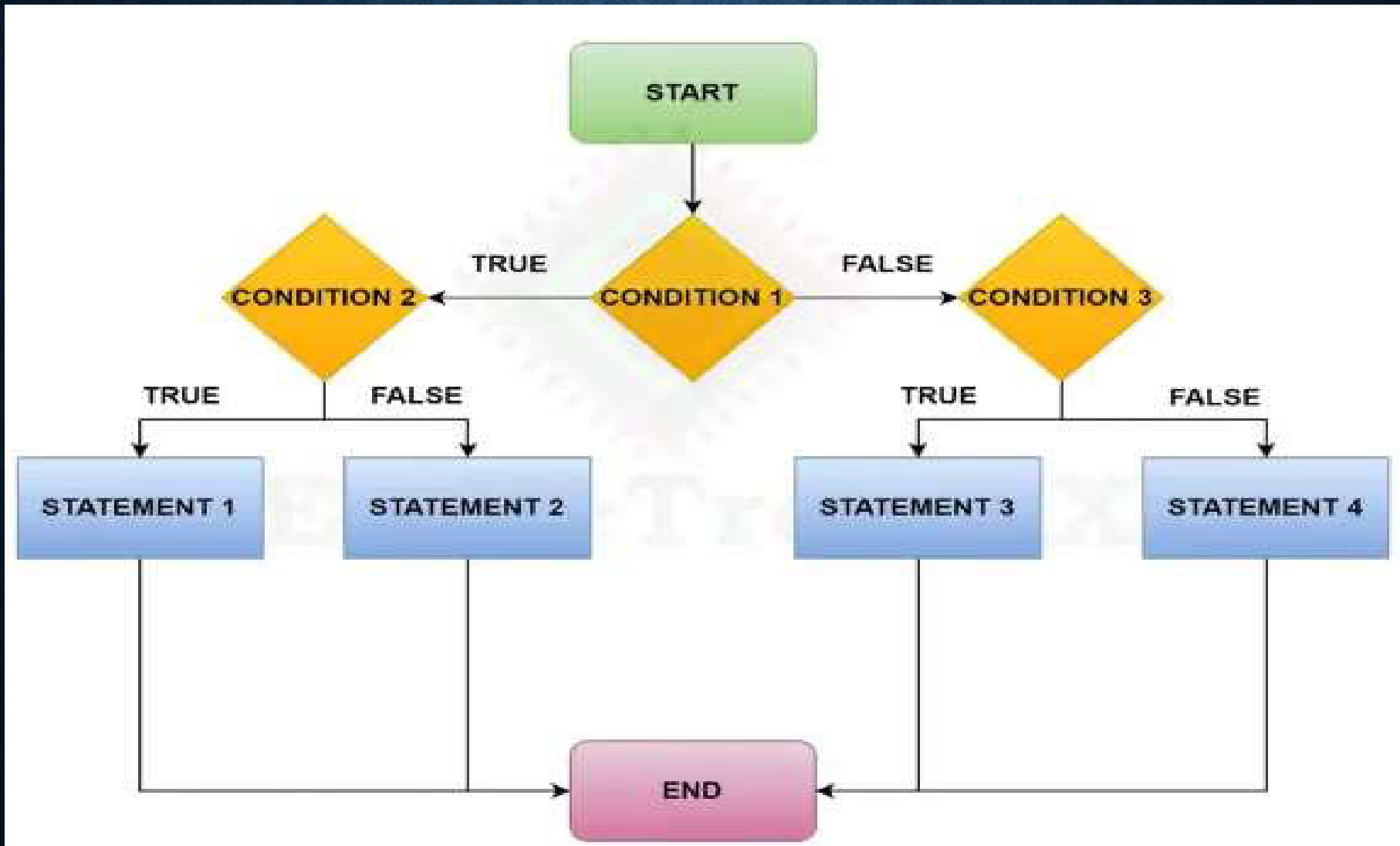
- Write a program to print whether the number is positive or negative using if else.

NESTED IF-ELSE STATEMENT

When a series of decisions are involved, we may have to use more than one if.....else statement in nested form.

```
if (condition 1)
{
   if (condition 2)
   {
      //statements 1
   }
   else
   {
      //statements 2
   }
}
else
{
   if (condition 3)
   {
      //statements 3
   }
   else
   {
      //statements 4
   }
}
```

NESTED IF-ELSE FLOWCHART



NESTED IF-ELSE PROGRAM

```c
#include<stdio.h>

int main()
{
   int n1, n2, n3;
   printf("Enter the numbers : ");
   scanf("%d %d %d", &n1, &n2, &n3);
   if(n1 > n2)
   {
      if(n1 > n3)
      {
         printf("[if inside if] %d is greater", n1);
      }
      else
      {
         printf("[else inside if] %d is greater", n3);
      }
   }
```

Program Continues

NESTED IF-ELSE PROGRAM

```
    else
    {
        if(n2 > n3)
        {
            printf("[if inside else] %d is greater", n2);
        }
        else
        {
            printf("[else inside else] %d is greater", n3);
        }
    }
}
```

NESTED IF-ELSE PROGRAM

Output 1

```
1.  Enter the numbers : 33 22 11
2.  [if inside if] 33 is greater
```

Output 2

```
1.  Enter the numbers : 33 11 44
2.  [else inside if] 44 is greater
```

Output 3

```
1.  Enter the numbers : 10 20 5
2.  [if inside else] 20 is greater
```

Output 4

```
1.  Enter the numbers : 11 22 33
2.  [else inside else] 33 is greater
```

NESTED IF-ELSE PROGRAM

- Write a program to print “You are eligible for ride based on age and height” using nested if else.

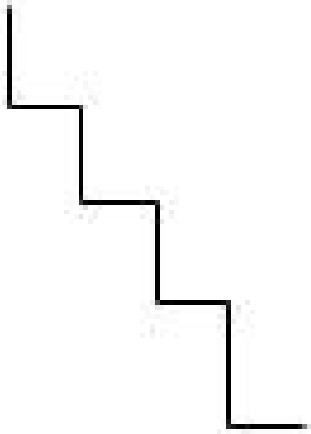
```
if (age >= 18){
        If (height >= 160){
        printf("You are eligible for the ride.\n");
        } else {
        printf("Sorry, you must be taller to go on the ride.\n");}
        }
else {
printf("Sorry, you must be at least 18 years old to go on the ride.\n");}
```

ELSE IF LADDER STATEMENT

- When a series of many conditions have to be checked we may use the ladder else if statement.
- The structure of else-if ladder looks like a ladder.

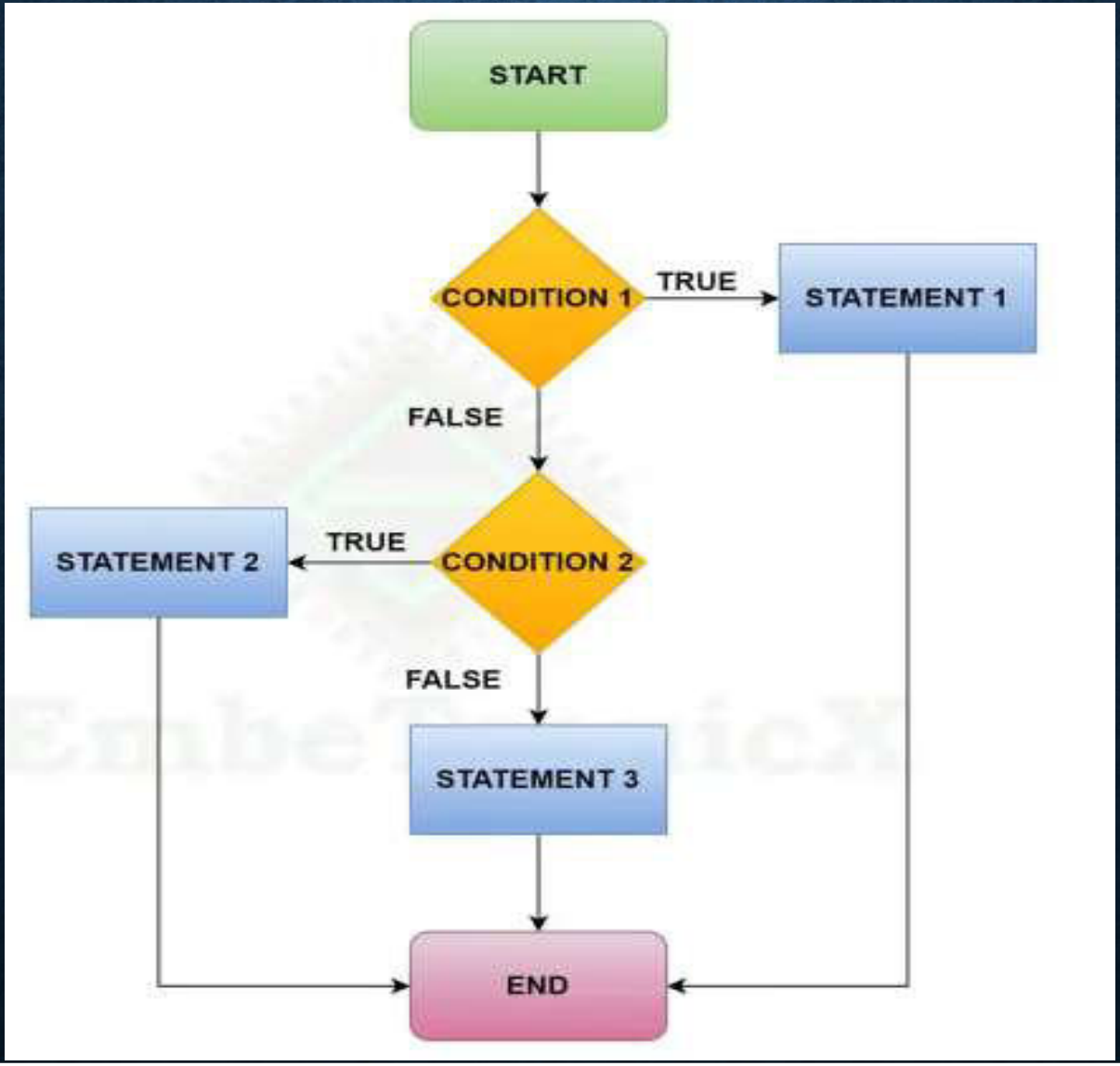
```
if (condition1)
{
   statement(s) 1;
}
else if (condition2)
{
   statement(s) 2;
}
else if (condition3)
{
   statement(s) 3;
}
else
{
   statement(s) 4;
}
```

ELSE IF LADDER STATEMENT

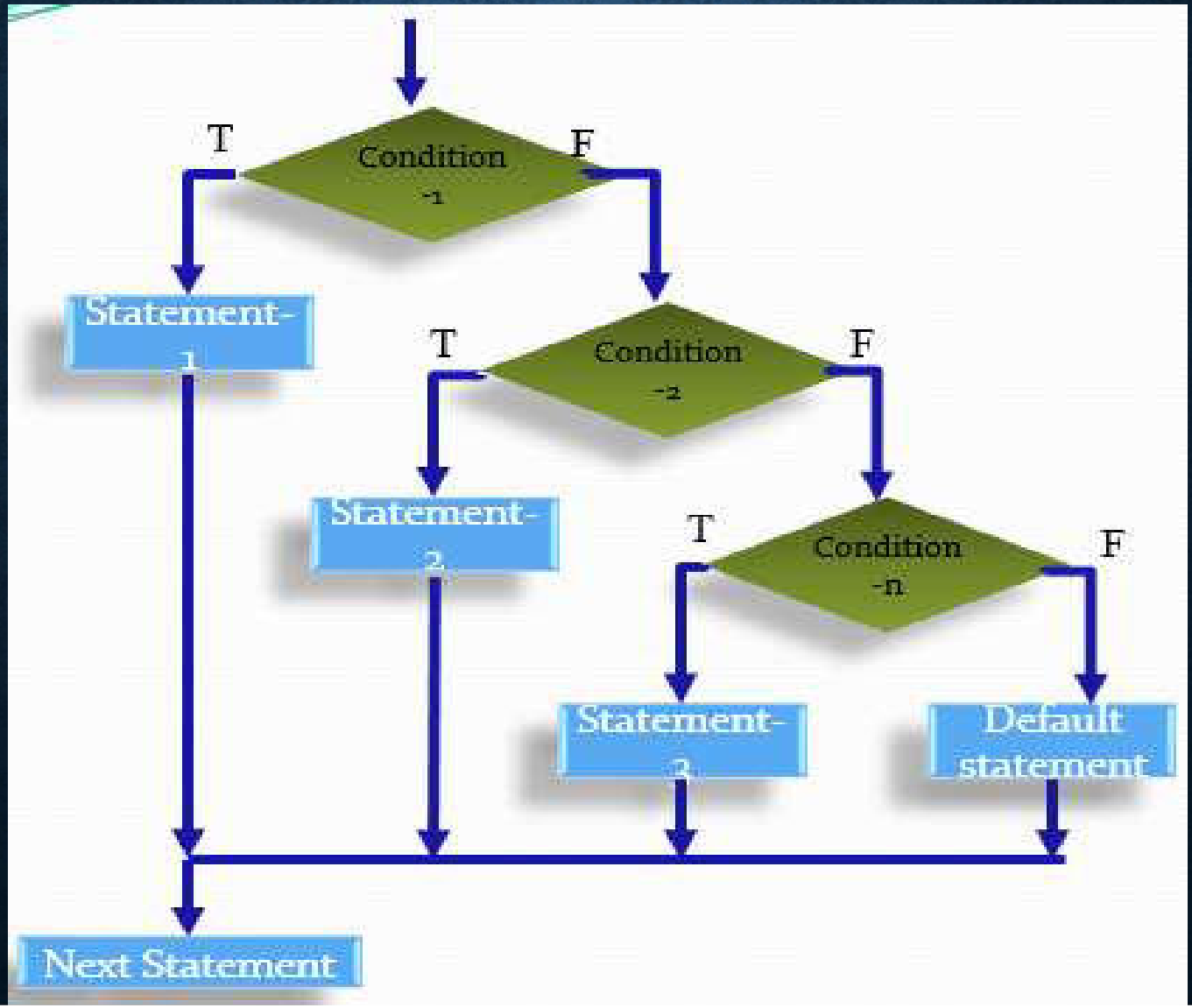


```
If (condition 1)
    statement-1;
else if (condition 2)
        statement-2;
    else if (condition n)
            statement-n;
        else
            default statement;
next statement;
```

ELSE IF LADDER FLOWCHART



ELSE IF LADDER FLOWCHART



ELSE IF LADDER PROGRAM

```
#include <stdio.h>

void main()
{
   int num1, num2;
   printf("Enter two numbers: ");
   scanf("%d %d", &num1, &num2);

   if(num1 == num2)
   {
      printf("Result is %d = %d",num1,num2);
   }

   else if (num1 > num2)
   {
      printf("Result is %d > %d", num1, num2);
   }

   else
   {
      printf("Result: %d < %d",num1, num2);
   }
}
```

ELSE IF LADDER PROGRAM

- Write a program to print respective class based on percentage using else if ladder. (Distinction – above 80%, First class – 70-80%, Second class – 60-70%, Fail - <50%)

```
#include <stdio.h>
void main() {
    float percentage;
    printf("Enter the percentage: ");
    scanf("%f", &percentage);

    if (percentage >= 80.0) {
        printf("Distinction\n");
    } else if (percentage >= 70.0 && percentage < 80.0) {
        printf("First Class\n");
    } else if (percentage >= 60.0 && percentage < 70.0) {
        printf("Second Class\n");
    } else if (percentage >= 50.0 && percentage < 60.0) {
        printf("Pass Class\n");
    } else {
        printf("Fail\n");
    }
}
```

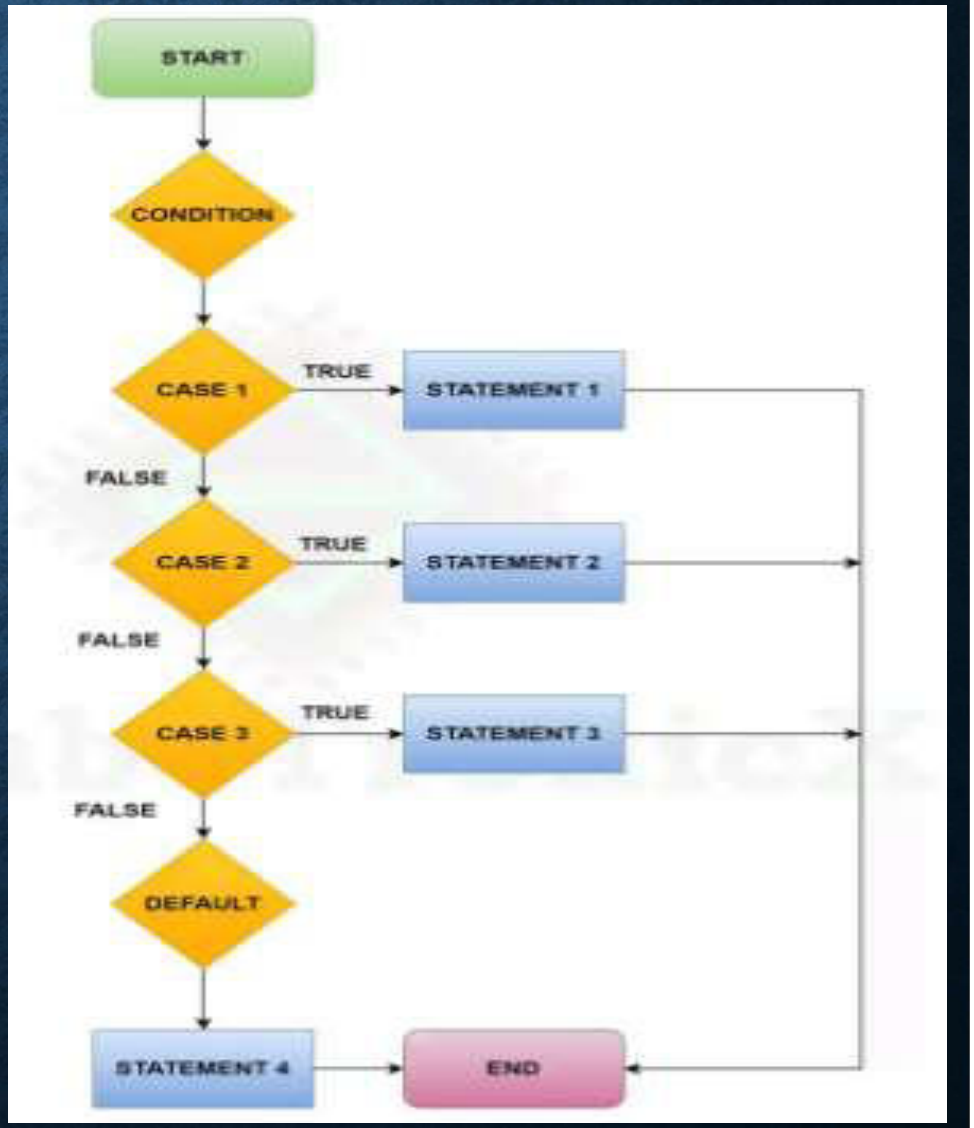
SWITCH CASE STATEMENT

- A switch case in the C programming language is a control structure that allows the execution of different pieces of code based on the value of a given expression.
- The switch case statement will allow multi-way of branching.

```
switch (selection)
{
   case LABEL1:
      statement(s) 1;
      break;
   case LABEL2:
      statement(s) 2;
      break;
   case LABEL3:
      statement(s) 3;
      break;
   default:
      statement(s) 4;
}
```

SWITCH CASE FLOWCHART

- **Note:**
- If we do not use the break statement in the case label, then it will execute the next case also even if the condition is not matching with the label.
- The values in case label can be constants, integers, characters only.
- The default case inside the switch statement is optional.



SWITCH CASE PROGRAM

```c
#include <stdio.h>

void main()
{
   int MyChoice;
   printf("Enter your choice: ");
   scanf("%d", &MyChoice);

   switch(MyChoice)
   {
      case 1:
         printf("You entered 1.\n");
         break;
      case 2:
         printf("You entered 2.\n");
         break;
      case 3:
         printf("You entered 3.\n");
         break;
      default:
         printf("Invalid choice.\n");
         break;
   }
}
```

SWITCH CASE PROGRAM1

```
#include <stdio.h>
void main() {
    char operator;
    int num1, num2, result;
    printf("Enter operator (+, -, *, /): ");
    scanf(" %c", &operator);
    printf("Enter two numbers: ");
    scanf("%d %d", &num1, &num2);
```

Program Continues

SWITCH CASE PROGRAM1

```
    switch(operator) {
        case '+':
                result = num1 + num2;
                break;
        case '-':
                result = num1 - num2;
                break;
        case '*':
                result = num1 * num2;
                break;
        case '/':
                result = (num2 != 0) ? num1 / num2 : 0;
                break;
        default:
                printf("Error: Invalid operator\n");
    }

    printf("Result: %d\n", result);
}
```

SWITCH CASE PROGRAM

```
#include <stdio.h>
void main()
{
int number=5;
switch (number)
{
   case 1:
   case 2:
   case 3:
     printf("One, Two, or Three.\n");
     break;
   case 4:
   case 5:
   case 6:
     printf("Four, Five, or Six.\n");
     break;
   default:
     printf("Greater than Six.\n");
}
}
```

SWITCH CASE PROGRAM

- Write a C program to read no 1 to 7 and print relatively day Sunday to Saturday. (Switch statement)

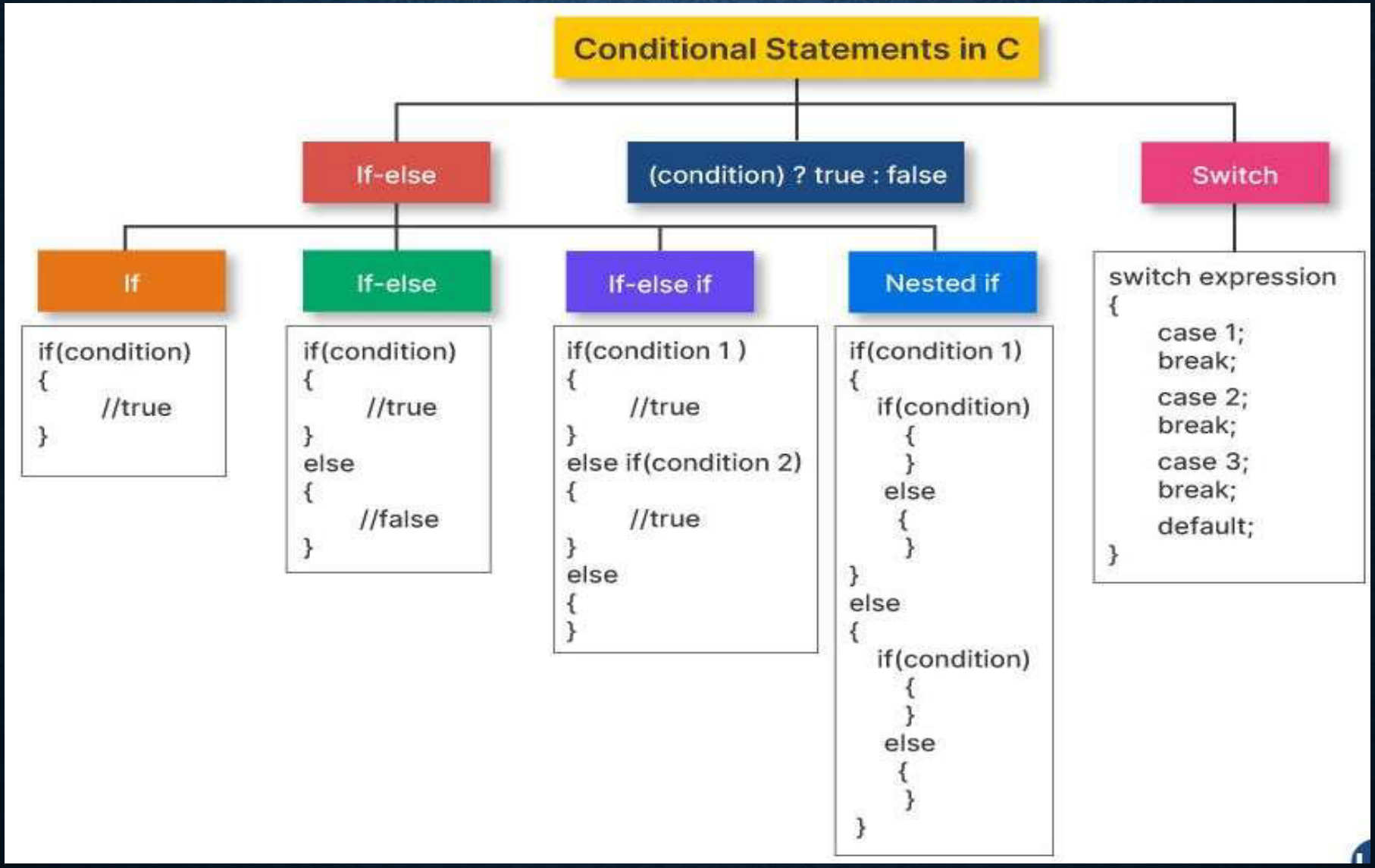
```
void main() {
   int dayNumber;

   printf("Enter a number (1 to 7): ");
   scanf("%d", &dayNumber);
```

Program Continues

SWITCH CASE PROGRAM

```
    switch(dayNumber) {
        case 1:
            printf("Monday\n");
            break;
        case 2:
            printf("Tuesday\n");
            break;
        case 3:
            printf("Wednesday\n");
            break;
        case 4:
            printf("Thursday\n");
            break;
        case 5:
            printf("Friday\n");
            break;
        case 6:
            printf("Saturday\n");
            break;
        case 7:
            printf("Sunday\n");
            break;
        default:
            printf("Invalid input! Please enter a number between 1 and 7.\n");
    }

}
```

SUMMARY


Conditional Statements in C
If-else
(condition) ? true : false
Switch
If
If-else
If-else if
Nested if
if(condition)
{
//true
}
if(condition)
{
//true
}
else
{
//false
}
if(condition 1)
{
//true
}
else if(condition 2)
{
//true
}
else
{
}
if(condition 1)
{
if(condition)
{
}
else
{
}
}
else
{
if(condition)
{
}
else
{
}
}
switch expression
{
case 1;
break;
case 2;
break;
case 3;
break;
default;
}

LOOPING STATEMENTS

1 While Loop

2 Do-While Loop

3 For Loop

4 Nested For Loop

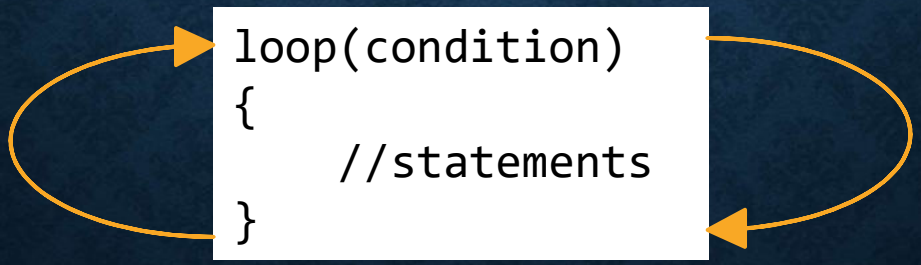
NEED OF LOOPING STATEMENTS

"Hello" 5

```
printf("Hello\n");
printf("Hello\n");
printf("Hello\n");
printf("Hello\n");
printf("Hello\n");
```

Output

```
Hello
Hello
Hello
Hello
Hello
```



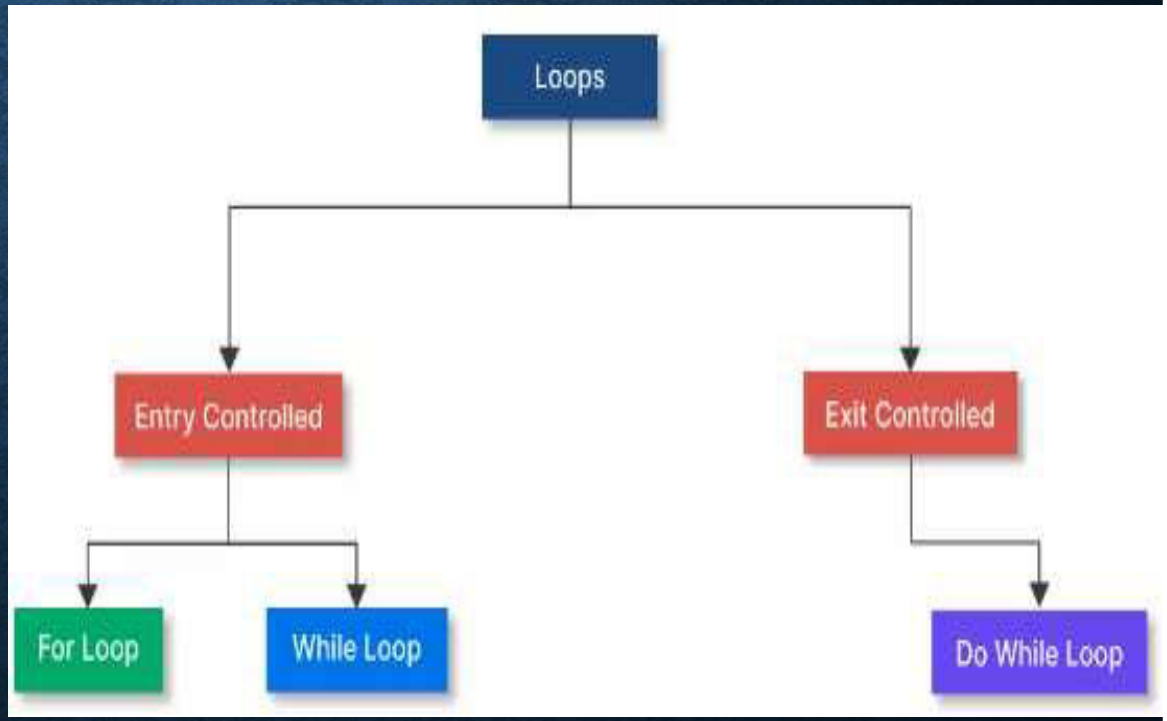
LOOPING STATEMENTS

- In looping, a sequence of statements are executed until some conditions for termination of the loop are satisfied.

- A program loop consist of two segments
  - Body of the loop
  - Control statement

LOOPING STATEMENTS

- Two types of control structure:
  - Entry-controlled loop
    1. For loop
    2. While loop
  - Exit-controlled loop
    3. Do-while loop
  - Virtual loop
    4. Go to statement



LOOPING STATEMENTS
FOR LOOP

For Loop has the following syntax:

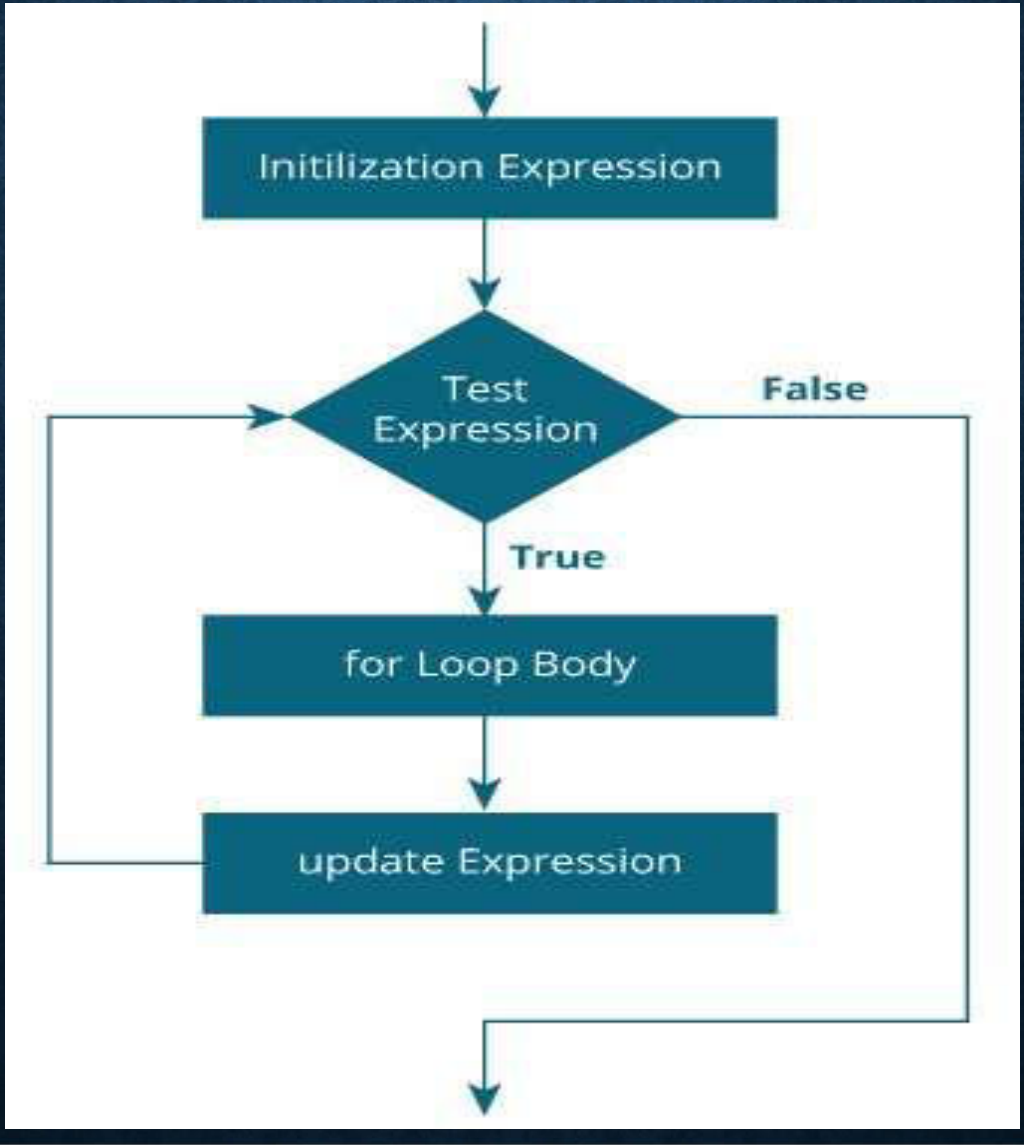
```
for (initialization; condition; increment/decrement) {

// code to be executed

}
```

LOOPING STATEMENTS
FOR LOOP

- Following are the steps in a looping process:

1. Initialization of loop control variable
2. Test of a specific condition for the execution of the loop
3. Execution of the statements in the body of the loop
4. Altering the value of the loop control variable

LOOPING STATEMENTS
FOR LOOP FLOWCHART



LOOPING STATEMENTS
FOR LOOP PROGRAM1

```
#include <stdio.h>

void main()
{
int z;

for (z = 0; z < 4; z++)
{
printf("Count: %d\n", z);
}
}
```

Output:

Cnt: 0
Cnt: 1
Cnt: 2
Cnt: 3

LOOPING STATEMENTS
FOR LOOP PROGRAM2

```c
#include <stdio.h>
void main()
{
    int num, count, sum = 0;
    printf("Enter a positive integer: ");
    scanf("%d", &num);

   for(count = 1; count <= num; ++count)
    {
        sum += count;
    }
    printf("Sum = %d", sum);
}
```

```
Enter a positive integer: 10
Sum = 55
```

LOOPING STATEMENTS
FOR LOOP FEATURES

Example:

```
p=1
for (i=0; i<5; i++)
```

The above statement can be written as,

```
for (p=1, i=0; i<5; i++)
```

Example:

```
for (n=1, m=10; n<=m; n++, m--)
```

Example:

```
for (i=0; i<10 && k==2; i++)
```

```
i=0;
for (; i<5;)
{
printf ("%d\n", i);
i++;
}
```

LOOPING STATEMENTS
FOR LOOP PROGRAM3

C Program: Print table for the given number using C for loop.

```
#include<stdio.h>
void main()
{
int i=1,number=0;
printf("Enter a number: ");
scanf("%d",&number);

for(i=1;i<=10;i++)
{
printf("%d \n",(number*i));
}

}
```

```
Enter a number: 2
2
4
6
8
10
12
14
16
18
20
```

LOOPING STATEMENTS
FOR LOOP PROGRAM4

Write a C program to find factorial of a given number.

```c
#include <stdio.h>

void main() {
    int n, i, factorial = 1;
    printf("Enter a number: ");
    scanf("%d", &n);

    if (n < 0) {
        printf("Factorial of a negative number doesn't exist.\n");
    } else {

        for(i = 1; i <= n; ++i) {
            factorial = factorial * i;
        }
        printf("Factorial of %d = %d\n", n, factorial);
    }
}
```

LOOPING STATEMENTS
FOR LOOP PROGRAM
GUESS THE OUTPUT?

```c
#include <stdio.h>
int main()
{
    int a,b,c;
    for(a=0,b=12,c=23;a<2;a++)
    {
        printf("%d ",a+b+c);
    }
}
```

Output

```
35  36
```

LOOPING STATEMENTS
FOR LOOP PROGRAM
GUESS THE OUTPUT?

```c
#include <stdio.h>
int main()
{
    int i=1;
    for(;i<5;i++)
    {
        printf("%d ",i);
    }
}
```

Output

```
1 2 3 4
```

LOOPING STATEMENTS FOR LOOP PROGRAM GUESS THE OUTPUT?

```c
#include <stdio.h>
int main()
{
    int i,j,k;
    for(i=0,j=0,k=0;i<4,k<8,j<10;i++)
    {
        printf("%d %d %d\n",i,j,k);
        j+=2;
        k+=3;
    }
}
```

Output

```
0 0 0
1 2 3
2 4 6
3 6 9
4 8 12
```

LOOPING STATEMENTS
FOR LOOP PROGRAM
GUESS THE OUTPUT?

```c
#include <stdio.h>
int main()
{
    int i;
    for(i=0;;i++)
    {
        printf("%d",i);
    }
}
```

Output

```
infinite loop
```

LOOPING STATEMENTS
FOR LOOP PROGRAM
GUESS THE OUTPUT?

```c
#include<stdio.h>
void main ()
{
    int i=0,j=2;
    for(i = 0;i<5;i++,j=j+2)
    {
        printf("%d %d\n",i,j);
    }
}
```

```
0 2
1 4
2 6
3 8
4 10
```

LOOPING STATEMENTS
FOR LOOP PROGRAM
GUESS THE OUTPUT?

```c
#include<stdio.h>
void main ()
{
    for(;;)
    {
        printf("welcome to computer dept.");
    }
}
```

```
welcome to computer dept.welcome to computer dept.welcome to computer dept
    .welcome to computer dept.welcome to computer dept.welcome to computer
    dept.welcome to computer dept.welcome to computer dept.welcome to
    computer dept.welcome to computer dept.welcome to computer dept
    .welcome to computer dept.welcome to computer dept.welcome to computer
    dept.welcome to computer dept.welcome to computer dept.welcome to
    computer dept.welcome to computer dept.welcome to computer dept
    .welcome to computer dept.welcome to computer dept.welcome to computer
    dept.welcome to computer dept.welcome to computer dept.welcome to
    computer dept.welcome to computer dept.welcome to computer dept
```

LOOPING STATEMENTS
NESTED FOR LOOP

- Nested for loop applications:
  - Multidimensional arrays / Matrices ,
  - Nested lists or trees,
  - Searching,
  - Sorting,
  - Graph traversal

Syntax

```
Outer_loop
{
 Inner_loop
 {
 // inner loop statements.
 }
 // outer loop statements.
}
```

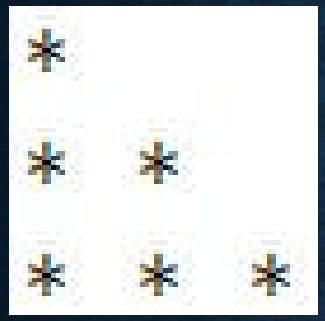
LOOPING STATEMENTS
NESTED FOR LOOP PROGRAM1

```
#include <stdio.h>
int main() {
 int i, j;
 for (i = 1; i <= 5; i++)
 {
   for (j = 1; j <= i; j++)
   {
   printf("%d ", j);
   }
 printf("\n");
 }
 return 0;
}
```

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

LOOPING STATEMENTS
NESTED FOR LOOP PROGRAM2

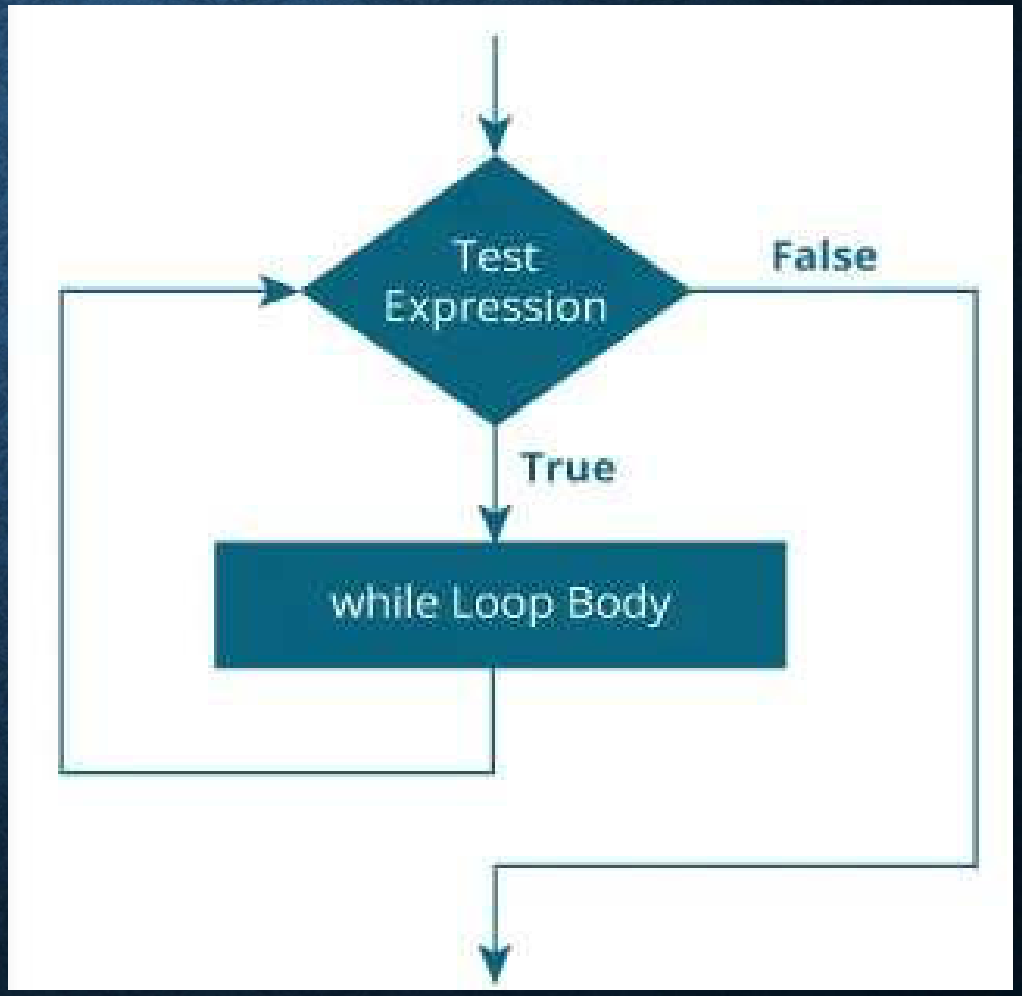
Write a program to print following pattern.




```c
#include <stdio.h>

int main() {
    int rows = 3;

    // Outer loop for rows
    for(int i = 1; i <= rows; i++) {

        // Inner loop for columns
        for(int j = 1; j <= i; j++) {
            printf("* ");
        }

        // Move to the next line after each row
        printf("\n");
    }

    return 0;
}
```

LOOPING STATEMENTS
WHILE LOOP

```
Syntax:
        while (Condition)
        {
            Statements;
        }
```



LOOPING STATEMENTS
NESTED WHILE LOOP

Syntax:

```
while (outer condition)
{
        Outer while Statements;

                while (inner condition)
                {
                        Inner while Statements;
                }

        Outer while Statements;
}
```

LOOPING STATEMENTS
WHILE LOOP PROGRAM1

```
#include <stdio.h>
int main() {
   int i = 1;

   while (i <= 5)
   {
      printf("%d\n", i);
      i++;
   }

   return 0;
}
```

```
1
2
3
4
5
```

LOOPING STATEMENTS
WHILE LOOP
GUESS THE OUTPUT?

```
#include <stdio.h>
int main()
{
      int var=1;
      while (var <=2)
      {
         printf("%d ", var);
      }
}
```

Output :
infinite
while loop

LOOPING STATEMENTS
WHILE LOOP
GUESS THE OUTPUT?

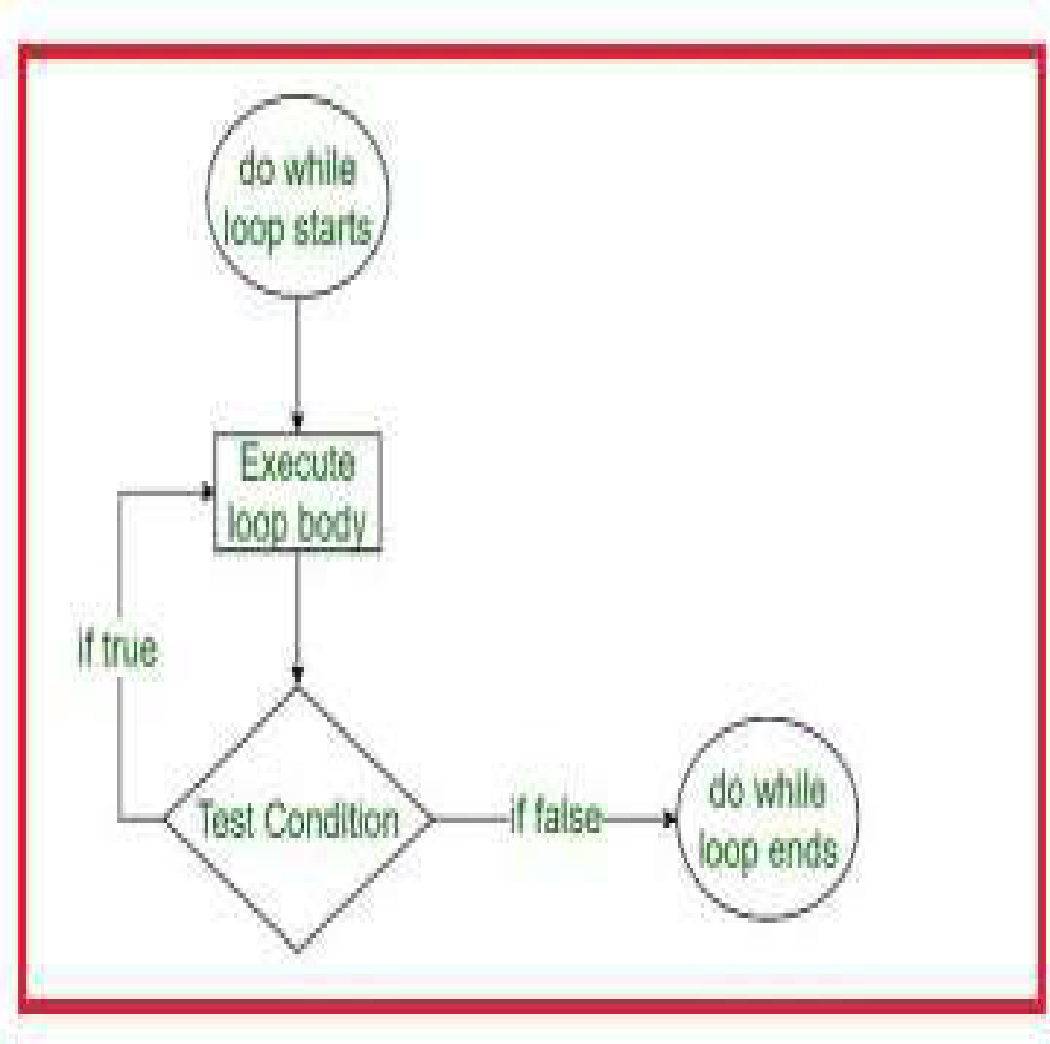
```
#include <stdio.h>
int main()
{
      int var = 6;
      while (var >=5)
      {
         printf("%d", var);
         var++;
      }
   return 0;
}
```

Output : infinite numbers starting from 6

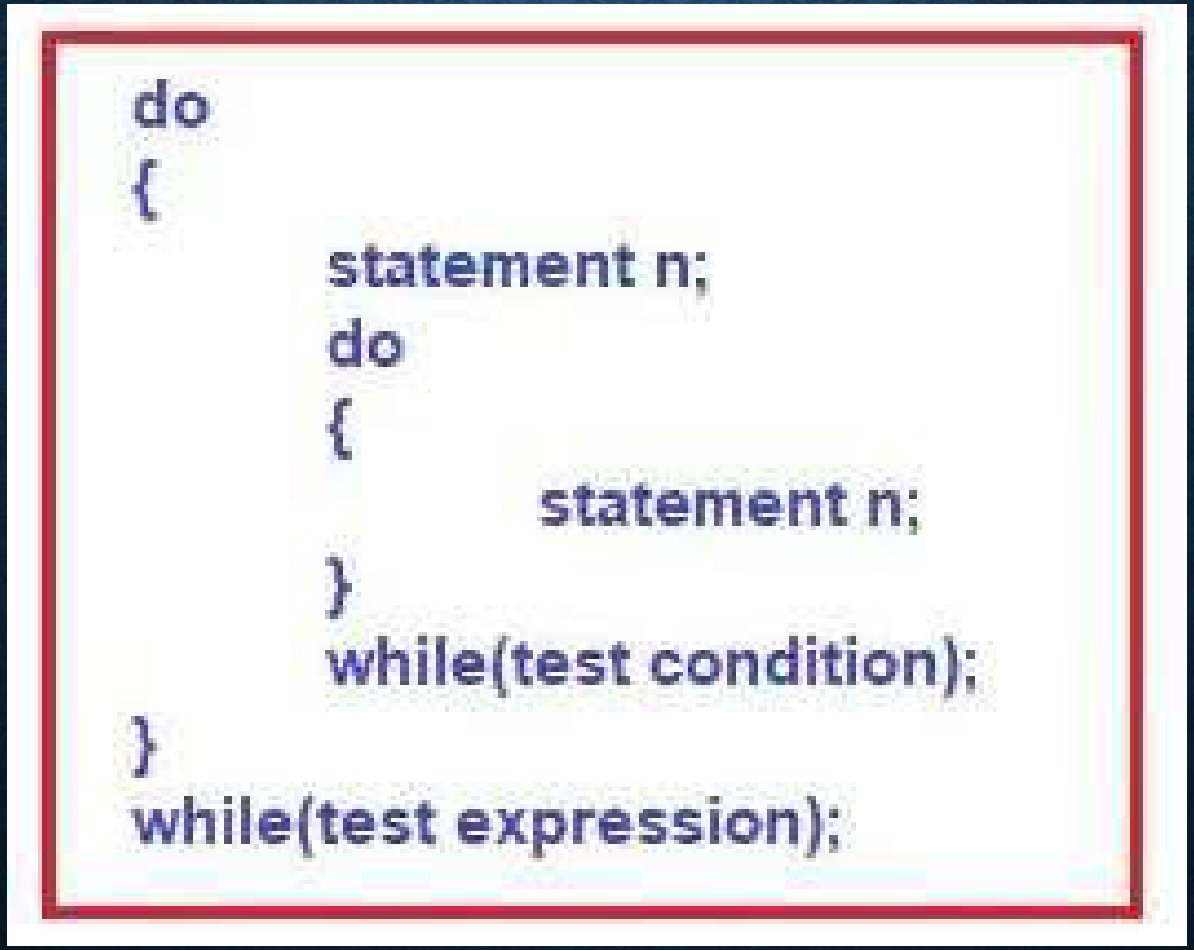
LOOPING STATEMENTS
DO WHILE LOOP

Syntax to use Do While

```
do
{
        statements;
}
while(condition);
```



LOOPING STATEMENTS
NESTED DO WHILE LOOP



```
do
{
        statement n;
        do
        {
                statement n;
        }
        while(test condition);
}
while(test expression);
```

LOOPING STATEMENTS
DO WHILE LOOP PROGRAM1

```
#include <stdio.h>
int main()
{
    int j=0;
    do
    {
        printf("Value of variable j is: %d\n", j);
        j++;
    }while (j<=3);
    return 0;
}
```

```
Value of variable j is: 0
Value of variable j is: 1
Value of variable j is: 2
Value of variable j is: 3
```

LOOPING STATEMENTS
DO WHILE LOOP PROGRAM2

```
#include <stdio.h>
int main(){
int i=1;
do{
printf("%d.",i);
printf("C programming ");
i=i+1;
}while(1);
}
```

```
1.C programming 2.C programming 3.C programming 4.C programming 5.C
programming 6.C programming 7.C programming 8.C programming 9.C
programming 10.C programming 11.C programming 12.C programming 13.C
programming 14.C programming 15.C programming 16.C programming 17.C
programming 18.C programming 19.C programming 20.C programming 21.C
```

SUMMARY

for loop	while statement	do-while statement
A for loop is used to execute and repeat a statement block depending on a condition which is evaluated at the beginning of the loop.	A while loop is used to execute and repeat a statement block depending on a condition which is evaluated at the beginning of the loop.	A do-while loop is used to execute and repeat a statement block depending on a condition which is evaluated at the end of the loop.
A variable value is initialized at the beginning of the loop and is used in the condition.	A variable value is initialized at the beginning or before the loop and is used in the condition.	A variable value is initialized before the loop or assigned inside the loop and is used in the condition.
A statement to change the value of the condition or to increment the value of the variable is given at the beginning of the loop.	A statement to change the value of the condition or to increment the value of the variable is given inside the loop.	A statement to change the value of the condition or to increment the value of the variable is given inside the loop.
The statement block will not be executed when the value of the condition is false.	The statement block will not be executed when the value of the condition is false.	The statement block will not be executed when the value of the condition is false, but the block is executed at least once irrespective of the value of the condition.
A for loop is commonly used by many programmers.	A while loop is widely used by many programmers.	A do-while loop is used in some cases where the conditions need to be checked at the end of the loop.

SUMMARY

Example:

```
for(i=1;i<=10;i++)
{
    s = s + i;
    p = p * i;
}
```

Example:

```
i =1;
while(i<=10)
{
    s = s + i;
    p = p * i;
    i++;
}
```

Example:

```
i =1;
do
{
    s = s + i;
    p = p * i;
    i++;
}
while(i<=10);
```

BREAK STATEMENT

- The break is a keyword in C which is used to bring the program control out of the loop. The break statement is used inside loops or switch statement.

BREAK STATEMENT

```
while (testExpression) {
    // codes
    if (condition to break) {
        break;
    }
    // codes
}
```

```
do {
    // codes
    if (condition to break) {
        break;
    }
    // codes
}
while (testExpression);
```

```
for (init; testExpression; update) {
    // codes
    if (condition to break) {
        break;
    }
    // codes
}
```

BREAK STATEMENT EXAMPLE1

```
#include <stdio.h>
void main()
{
    int i;
    double number, sum = 0.0;

    for (i = 1; i <= 10; ++i)
          {
           printf("Enter n%d: ", i);
           scanf("%lf", &number);

          // if the user enters a negative number,
                                    break the loop
           if (number < 0.0)
           {
            break;
           }

        sum += number;
          }
    printf("Sum = %f", sum);
}
```

```
Enter n1: 2.4
Enter n2: 4.5
Enter n3: 3.4
Enter n4: -3
Sum = 10.30
```

BREAK STATEMENT EXAMPLE2

```
#include<stdio.h>
int main(){
 int i,j;
 for(i=1;i<=3;i++)
 {
     for(j=1;j<=3;j++)
     {
     printf("%d %d\n",i,j);
     if(i==2 && j==2)
     {
     break;
     }
     }
}
return 0;
}
```

```
1 1
1 2
1 3
2 1
2 2
3 1
3 2
3 3
```

BREAK STATEMENT EXAMPLE3

```
#include<stdio.h>
void main ()
 {
 int i = 0;
 while(1)
 {
 printf("%d ",i);
 i++;
 if(i == 10)
 break;
 }
 printf("came out of while loop");
}
```

```
0 1 2 3 4 5 6 7 8 9 came out of while loop
```

BREAK STATEMENT EXAMPLE4

```
#include<stdio.h>
void main ()
 {
 int n=2,i,choice;
 do
 {
 i=1;
   while(i<=10)
   {
    printf("%d X %d = %d\n",n,i,n*i);
    i++;
   }
 printf("do you want to continue with the table of %d ,
          enter any nonzero value to continue.",n+1);
 scanf("%d",&choice);
 if(choice == 0)
 {
 break;
 }
 n++;
 }while(1);
}
```

CONTINUE STATEMENT

- The continue statement is used to bypass the remainder of the current pass through a loop → tells the compiler **"Skip the following Statements and continue with the next Iteration".**

- In while and do loops → Continue Go directly to the test – condition and then to continue the iteration process.

- In the case of for loop → The updation section of the loop is executed before test-condition, is evaluated.

CONTINUE STATEMENT

```
while (Test condition)
 {
- - - - - - - - -
if ( - - - - - - - -)
  continue;
   ---------------------
   ---------------------
 }
```

CONTINUE STATEMENT

```
do
 {
- - - - - - - -
if ( - - - - - - -)
 continue;
  ------------------
  ------------------
} while (Test condition);
```

CONTINUE STATEMENT

```
for(initialization; test condition; increment)
    {
  - - - - - - - -
  if ( - - - - - - - -)
    continue;
     ------------------
     ------------------
```

CONTINUE STATEMENT EXAMPLE1

```c
#include <stdio.h>
void main() {
    int i;
    double number, sum = 0.0;

    for (i = 1; i <= 5; ++i)
    {
        printf("Enter a n%d: ", i);
        scanf("%lf", &number);
        if (number < 0.0)
        {
            continue;
        }

        sum += number;
    }
    printf("Sum = %lf", sum);
}
```

```
Enter a n1: -5
Enter a n2: -6
Enter a n3: -7
Enter a n4: -2
Enter a n5: -7
Sum = 0.000000
```

```
Enter a n1: 1
Enter a n2: 2
Enter a n3: 3
Enter a n4: -5
Enter a n5: 5
Sum = 11.000000
```

CONTINUE STATEMENT EXAMPLE2

```
#include<stdio.h>
 int main()
 {
 int i=1,j=1;
 for(i=1;i<=3;i++)
 {
 for(j=1;j<=3;j++)
 {
 if(i==2 && j==2)
 {
 continue;//will continue loop of j only
 }
 printf("%d %d\n",i,j);
}
}
return 0;
}
```

```
1 1
1 2
1 3
2 1
2 3
3 1
3 2
3 3
```

CONTINUE STATEMENT EXAMPLE3

```
#include<stdio.h>
 void main ()
 {
 int i = 0;
 while(i!=10)
 {
 printf("%d", i);
 continue;
 i++;
 }
}
```

```
0000000000000000000000000000000000000000000000000000000000000000000000000000
 0000000000000000000000000000000000000000000000000000000000000000000000000000
 0000000000000000000000000000000000000000000000000000000000000000000000000000
 0000000000000000000000000000000000000000000000000000000000000000000000000000
 0000000000000000000000000000000000000000000000000000000000000000000000000000
 0000000000000000000000000000000000000000000000000000000000000000000000000000
 0000000000000000000000000000000000000000000000000000000000000000000000000000
```

Break and Continue Program

```
#include<stdio.h>
void main()
{
int i = 1, j = 0;
while(i<=5)
{
 i=i+1;
  if(i== 2)
  { continue;   }
    j=j+1;
    }
printf("value of i=%d and j=%d",i,j);
}
```

```
value of i=6 and j=4
```

```
#include<stdio.h>
void main()
{
int i = 1, j = 0;
while(i<=5)
{
 i=i+1;
  if(i== 2)
  { break;   }
    j=j+1;
    }
printf("value of i=%d and j=%d",i,j);
}
```

```
value of i=2 and j=0
```

No.	break statement	continue statement
1.	A **break** statement is used to terminate the execution of a statement block. The next statement which follows this statement block will be executed. The keyword is **break;**	A **continue** statement is used to transfer the control to the beginning of a statement block. The keyword is **continue;**
2.	When a **break** statement is executed in a loop, a repetition of the loop will be terminated.	When a **continue** statement is executed in a loop, the execution is transferred to the beginning of the loop.

GOTO STATEMENT

- The goto statement is known as jump statement/virual looping in C.

- As the name suggests, goto is used to transfer the program control to a predefined label.

- ***It can also be used to break the multiple loops which can't be done by using a single break statement.*** However, using goto is avoided these days since it makes the program less readable and complicated.

GOTO STATEMENT

Syntax:

```
Syntax1          |   Syntax2
-----------------------------
goto label;      |    label:
.                |     .
.                |     .
.                |     .
label:           |    goto label;
```

GOTO STATEMENT PROGRAM1

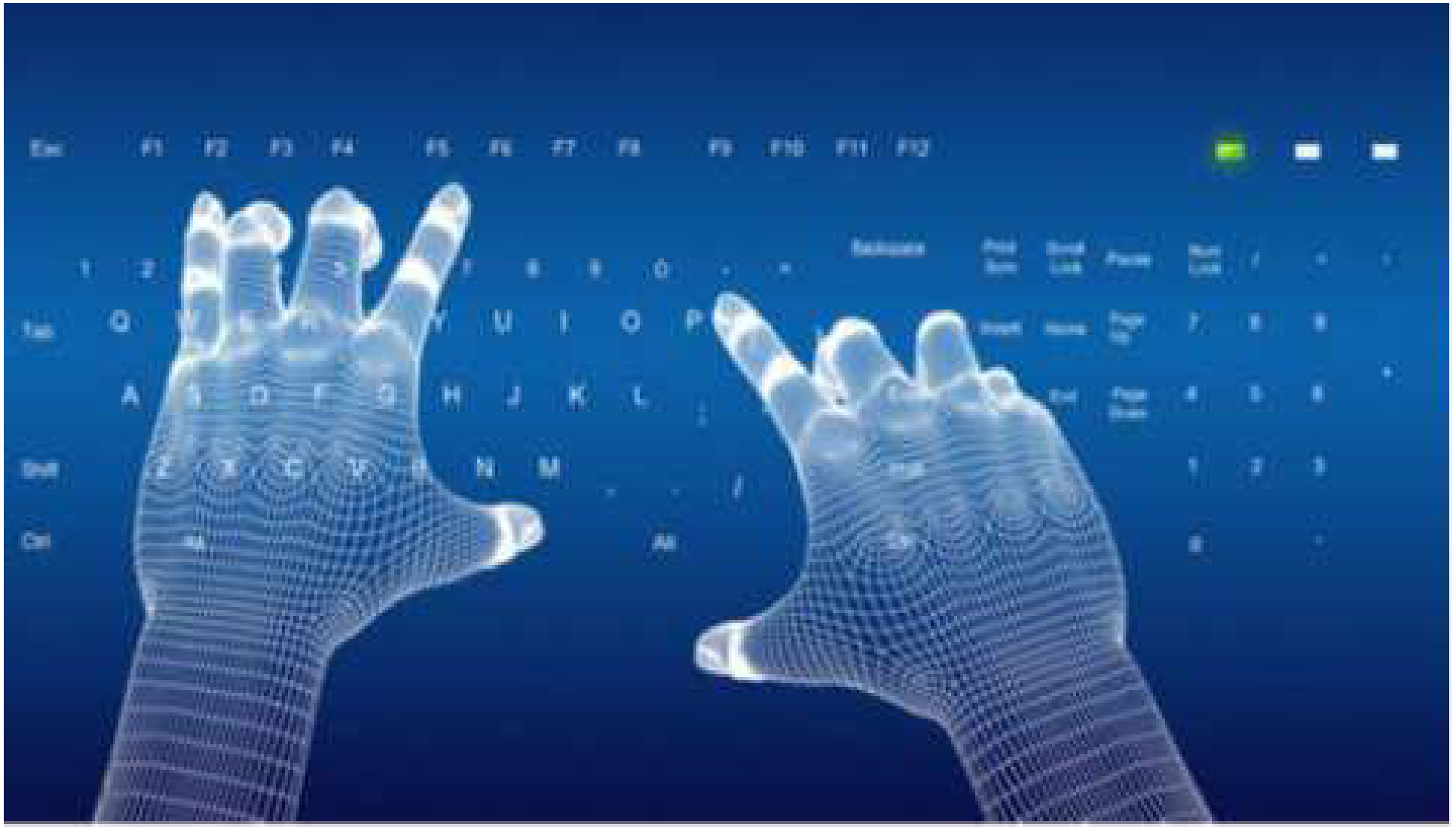
```c
#include <stdio.h>
int main()
{
 int num,i=1;
 printf("Enter the number whose table you want to print?");
 scanf("%d",&num);

 table:
   printf("%d x %d = %d\n",num,i,num*i);
   i++;
   if(i<=10)
  goto table;
}
```

```
Enter the number whose table you want to print?22
22 x 1 = 22
22 x 2 = 44
22 x 3 = 66
22 x 4 = 88
22 x 5 = 110
22 x 6 = 132
22 x 7 = 154
22 x 8 = 176
22 x 9 = 198
22 x 10 = 220
```

GOTO STATEMENT PROGRAM2

```
#include <stdio.h>
int main()
{
 int i, j, k;
 for(i=0;i<10;i++)
 {
 for(j=0;j<5;j++)
 {
 for(k=0;k<3;k++)
  {
  printf("%d %d %d\n",i,j,k);
  if(j == 3)
   {
   goto out;
   }
  }
 }
 }
out:
 printf("came out of the loop");
}
```

```
0 0 0
0 0 1
0 0 2
0 1 0
0 1 1
0 1 2
0 2 0
0 2 1
0 2 2
0 3 0
came out of the loop
```



Thank You !